Prueba de laboratorio: transmisión de archivos con HackRF

Sistemas de Comunicaciones Digitales – FACET – UNT

Guía para la última parte del proyecto integrador

---

1. Qué se va a hacer

Se transmite un archivo (texto, imagen, lo que sea) de un HackRF a otro, **con el mismo sistema que se simuló**:

- las mismas tramas: preámbulo de 64 bits, longitud, datos y CRC-32;
- la misma modulación: 4-QAM con codificación diferencial y filtro RRC;
- el mismo receptor: filtro adaptado, sincronismo de símbolo, lazo de Costas y decodificación diferencial.

Al final se comprueba que el archivo llegó **idéntico**, comparando su huella SHA-256 con la del transmisor. Además se miden el desvío entre los osciladores de los dos HackRF, el Eb/N0 y la tasa de tramas perdidas (PER).

Half-duplex. El HackRF es un equipo half-duplex: puede transmitir o recibir, pero no las dos cosas a la vez. En cada prueba un nodo transmite y el otro recibe. Después se invierten los papeles (A → B y luego B → A), igual que los turnos del sistema simulado.

Todo está armado **solo con bloques estándar de GNU Radio**. Los bloques `Soapy HackRF Sink/Source` y el driver del HackRF ya vienen con RadioConda, así que no hay que instalar nada más.

Archivo	Qué es
`grc_hackrf/hackrf_tx_archivo.grc`	Transmisor: lee el archivo, arma los paquetes y los transmite por el HackRF
`grc_hackrf/hackrf_rx_archivo.grc`	Receptor: recibe con el HackRF, guarda los paquetes con CRC correcto y muestra los gráficos
`reconstruir_archivo.py`	Arma el archivo recibido, cuenta los paquetes perdidos y verifica la huella SHA-256
`grc_hackrf/archivo_a_enviar.txt`	Archivo de ejemplo que se transmite por defecto
`tests/test_hackrf.py`	Prueba todo **sin HackRF**, con un canal de laboratorio simulado
`hackrf_simulada/`	Ensayo completo del laboratorio en una sola PC, con un emulador de las dos HackRF (ver su `LEEME.md`)

2. Material

- 2 HackRF One con sus cables USB.
- 2 antenas para la banda elegida, o un cable SMA con **atenuador de 30 dB o más**.
- 1 o 2 computadoras con RadioConda (GNU Radio 3.10). Con una sola PC se conectan los dos HackRF a la misma máquina.

3. Seguridad: leer antes de encender

- **Frecuencia.** Usar la frecuencia que autorice la cátedra. Por defecto los `.grc` usan **915 MHz**, que está dentro de la banda ISM de 902–928 MHz (Región 2), y la potencia más baja que alcance.
- **Nunca unir el TX y el RX con un cable directo, sin atenuador.** La entrada del HackRF soporta como máximo −5 dBm, y la salida puede llegar a +10 dBm o más con el amplificador encendido: se puede dañar el receptor.

- Empezar siempre con los **amplificadores apagados** (casillas "Amplificador TX/RX") y ganancias bajas.
- Conectar la antena o el cable antes de transmitir.

4. Antes del laboratorio (en casa, sin HackRF)

1. Correr la prueba que simula el laboratorio: `python tests/test_hackrf.py`. Tienen que salir **7 pruebas, 0 fallas**, en menos de un minuto. La prueba usa los `.grc` reales y reemplaza los HackRF por un canal que imita al laboratorio: desvío de hasta ±40 kHz entre los osciladores, relojes de muestreo distintos en ±20 ppm, ruido de fase, atenuación, portadora residual y ruido antes de que empiece la transmisión.
2. Ensayar el procedimiento completo con la **HackRF simulada** (`hackrf_simulada/LEEME.md`). Un programa hace de las dos placas y del aire. Se abren en GNU Radio Companion las versiones simuladas del TX y del RX, que son iguales a las del laboratorio salvo el bloque de la placa. Se transmite un archivo y se lo reconstruye, como en las secciones 6.1 a 6.6. La guía trae 9 ejercicios (ganancias, saturación, frecuencias distintas, antenas lejos, cable sin atenuador). `python hackrf_simulada/prueba_completa.py` los corre solos en ~40 s.
3. Abrir los dos `.grc` en GNU Radio Companion y verificar que no haya bloques en rojo.
4. Elegir el archivo a enviar y copiarlo a la carpeta `grc_hackrf/`. Conviene uno de menos de 200 kB, por ejemplo un texto o una foto chica. Si está en otra carpeta, escribir la ruta completa en el parámetro `archivo`. Un archivo de 1 MB tarda unos 17,5 s por pasada.

5. Conectar y reconocer los HackRF

En el *radioconda Prompt*:

```
hackrf_info
```

Debe listar cada placa con su número de serie (*Serial number*). Si no aparece, probar otro cable o puerto USB. En Windows 10/11 normalmente el driver se instala solo; si no, instalar el driver WinUSB con Zadig.

Si los dos HackRF están en la misma computadora, hay que decirle a cada `.grc` cuál usar:

1. Correr `SoapySDRUtil --find="driver=hackrf"` y anotar el `serial = ...` de cada placa.
2. En cada `.grc`, abrir la variable `dispositivo` y escribir, por ejemplo, `"serial=0000000000000000a06063c8234e5f5f"`. En el transmisor va un serial y en el receptor el otro.

Revisar cada placa antes de transmitir. Sirve para encontrar una placa con problemas antes de la prueba, y no hace falta la otra placa:

1. `hackrf_info` tiene que decir **HackRF One** y mostrar la versión de firmware (*Firmware Version*).
2. Abrir `hackrf_rx_archivo.grc`, escribir **98** en "Frecuencia [MHz]" y apretar Enter. En "Espectro del HackRF" y en la cascada se tienen que ver las radios FM de la zona: la placa recibe bien. Después volver a 915.
3. Mientras corre un `.grc`, se prende el LED **RX** (recibiendo) o **TX** (transmitiendo) de la placa.

La antena (o el cable con atenuador) va en el conector **ANTENNA**, el que está en la punta opuesta al USB. Los dos conectores SMA que están junto al USB son **CLKIN** y **CLKOUT**: no son para la antena (ver la prueba 4 opcional en la sección 7).

6. Procedimiento

6.1 Encender primero el receptor

Abrir `hackrf_rx_archivo.grc` y ejecutarlo (F6). En la pestaña **1. Señal del HackRF** se ve el piso de ruido y una espiga angosta en el centro: es la portadora residual del oscilador del HackRF, y es normal. El receptor **sintoniza**

400 kHz por debajo de la frecuencia elegida, así que la señal útil va a aparecer a la derecha de la espiga (+400 kHz) y no queda tapada por ella.

6.2 Encender el transmisor

1. Abrir `hackrf_tx_archivo.grc`.
2. En el parámetro `archivo` poner el nombre del archivo. En `repeticiones` poner cuántas veces se envía (3 por defecto; 0 = sin fin).
3. Ejecutarlo (F6). Arriba se ven el nombre, el tamaño, los paquetes por pasada, la duración de cada pasada y la huella SHA-256 (los primeros 16 caracteres).

El archivo se envía en **pasadas**:

```
[relleno 2500 bytes | paquete 0 | paquete 1 | ... | último paquete con nombre, tamaño y huella] x repeticiones
```

El relleno del comienzo le da tiempo al receptor para engancharse. Si un paquete se pierde en una pasada, se recupera en la siguiente: el receptor no puede pedir que se repita, porque no hay canal de vuelta.

6.3 Ajustar las ganancias

Mirar en el receptor:

- **"Espectro del HackRF"**: la señal debe quedar entre **20 y 40 dB por encima del ruido**.
- **"Señal cruda del HackRF"**: la señal debe quedar **lejos de ±1**, como mucho alrededor de 0,5. Si toca ±1, el HackRF se satura: hay que bajar las ganancias.
- En el transmisor, dejar la **"Amplitud digital" en 0,6 o menos**. Con 0,5 los picos de I y Q llegan a 0,76; con 0,7 pasan de ±1 y el DAC de 8 bits recorta la señal (se comprobó con la HackRF simulada).

Para subir la señal, el orden es: VGA del RX → LNA del RX → VGA del TX. Los amplificadores se encienden solo si no alcanza, y nunca con cable sin atenuador.

6.4 Verificar que se reciben los paquetes

En la pestaña **2. Sincronismo**:

- "Desvío de frecuencia que estima el FLL" debe quedar quieto, en unos pocos kHz.
- En la constelación, "Antes" es un anillo (la fase gira) y "Después" son 4 nubes (Costas corrigió la fase).

En los indicadores de arriba:

- "Tramas detectadas" y "Con CRC correcto" van subiendo juntos.
- "Descartadas por CRC" queda en 0 o casi 0.

En la pestaña **3. Paquetes recibidos**, cada fila de puntos es un número de paquete: un punto que falta es un paquete perdido.

Anotar el desvío de frecuencia [Hz] y [ppm] y el Eb/N0 estimado.

6.5 Terminar y reconstruir el archivo

1. Esperar a que el transmisor termine: "Pasadas enviadas" llega al número de repeticiones. Cerrar las dos ventanas con la **X**.
2. Correr `python reconstruir_archivo.py`. Lee `grc_hackrf/paquetes_recibidos.bin`, escribe `grc_hackrf/recibido_<nombre>` y muestra cuántos paquetes llegaron y cuántos se perdieron, el PER medido y si la huella SHA-256 es **IGUAL** a la del transmisor (o sea, si el archivo llegó idéntico).
3. Abrir `recibido_<nombre>` y comprobarlo a simple vista.

4. **Copiar los resultados a otra carpeta** con el nombre de la prueba: `paquetes_recibidos.bin`, `datos_recibidos.bin` y `recibido_*`. La próxima vez que corra el receptor, los pisa.

6.6 Invertir los papeles

Cambiar qué nodo transmite y cuál recibe (en dos PCs, intercambiar los `.grc`) y repetir: así queda probado el enlace en los dos sentidos, uno por vez (half-duplex).

7. Mediciones para el informe

Prueba	Sentido	Distancia o cable	Ganancias TX (VGA/amp) y RX (LNA/VGA/amp)	Eb/N0 estimado	Desvío [Hz] / [ppm]	Paquetes recibidos / perdidos	PER medido	PER teórico	¿Idéntico?
1. Señal buena	A → B								
2. Señal débil	A → B								
3. Señal buena	B → A								
4. Reloj compartido (opcional)	A → B								

- **Prueba 1, señal buena:** es lo normal a pocos metros. El archivo debe llegar idéntico, con PER ≈ 0.
- **Prueba 2, señal débil:** bajar la VGA del TX (o alejar los equipos) hasta que aparezcan pérdidas, con un Eb/N0 estimado de 8 a 10 dB. Comparar el PER medido (`reconstruir_archivo.py`) con el "PER teórico" del indicador, y mostrar que con varias repeticiones el archivo igual llega completo.
- **Prueba 3:** el enlace en el otro sentido.
- **Prueba 4, reloj compartido (opcional):** las HackRF One tienen una entrada (**CLKIN**) y una salida (**CLKOUT**) de reloj de 10 MHz, junto al USB. Si las dos placas usan el mismo reloj, el desvío entre osciladores tendría que bajar casi a 0 Hz:
  1. En la placa que da el reloj (la del TX), habilitar la salida: `hackrf_clock -o 1 -d <serial del TX>`. En los firmware recientes viene apagada.
  2. Unir con un cable SMA el CLKOUT de esa placa con el CLKIN de la otra.
  3. Comprobar con `hackrf_clock -i -d <serial del RX>` que la otra placa detecta el reloj. La placa empieza a usarlo recién cuando arranca a transmitir o recibir, así que hay que volver a ejecutar los `.grc` después de conectar.
  4. Repetir la prueba 1 y comparar el "Desvío de frecuencia" con el de la prueba 1. La diferencia es justamente lo que corrigen el FLL y el lazo de Costas cuando los relojes son distintos.
- **Velocidad útil:** tamaño del archivo ÷ duración de una pasada. Por ejemplo, 2089 bytes en 0,08 s ≈ 200 kbit/s. Con archivos grandes se acerca a 450 kbit/s; el resto se va en encabezados, CRC y relleno.
- **Desvío entre osciladores en ppm:** es la diferencia entre los cristales de los dos HackRF. La especificación admite hasta ±20 ppm por placa, y el indicador lo calcula como desvío [Hz] ÷ frecuencia [MHz].
- **Opcional, grabar la señal para analizarla en casa:** en el receptor, habilitar el bloque `grabar_iq` (clic derecho → Enable). Ocupa 16 MB por segundo. Para analizar la grabación sin HackRF: deshabilitar `hackrf_rx` y habilitar `leer_grabacion` y `acelerador`.

8. Cómo funciona (para poder explicarlo)

8.1 Qué cambia respecto de la simulación

Problema del hardware real	Solución en los `.grc`
El receptor no sabe a qué parte del archivo pertenece cada paquete, ni si se perdió alguno	Cada paquete lleva un **número** de 2 bytes y la pasada termina con un **paquete de descripción** del archivo: nombre, tamaño, bytes por paquete, cantidad de paquetes y huella SHA-256
Al encender, los lazos del receptor (AGC, FLL, sincronismo, Costas) tardan en engancharse	**Relleno** al comienzo de cada pasada: 2500 bytes al azar, sin preámbulo, que el receptor descarta
El HackRF tiene una espiga de continua (portadora residual) justo en la frecuencia sintonizada	**Desplazamiento digital de 400 kHz**: el TX corre la señal +400 kHz (`Rotator`) y los dos HackRF se sintonizan 400 kHz más abajo. El RX la vuelve a centrar y filtra (`Frequency Xlating FIR Filter`), con lo que la espiga queda afuera
El HackRF trabaja a 2 M muestras/s como mínimo	TX con 8 muestras/símbolo (250 000 símbolos/s = 500 kbit/s). El RX diezma a 1 M muestras/s y vuelve a tener 4 muestras/símbolo: **el mismo receptor que la simulación**
El nivel recibido depende de la distancia y las ganancias	**AGC** (`AGC2`): normaliza la amplitud antes de los lazos
Los osciladores de dos HackRF difieren hasta ±20 ppm cada uno: decenas de kHz a 915 MHz	**FLL band-edge**: corrige el desvío grueso (kHz) usando los bordes del espectro RRC. El lazo de Costas solo corrige lo que queda (cientos de Hz) y la fase
Hay paquetes perdidos y no hay canal de vuelta para pedirlos	**Repeticiones** del archivo; `reconstruir_archivo.py` toma cada paquete de la pasada en que llegó bien

8.2 Anchos de banda de los lazos

Se eligieron con el canal de laboratorio simulado, con Es/N0 = 12 dB (Eb/N0 = 9 dB) y desvío de ±40 kHz:

FLL	Costas	PER medido	PER teórico
0,005	0,01	0,063 – 0,090	0,058
0,01	0,01	0,21 – 0,23	0,058
0,01	0,02	0,07 – 0,10	0,058
0,02	0,02	0,39 – 0,41	0,058

Un FLL más ancho se engancha más rápido pero deja pasar más ruido. Con 0,005 se engancha durante el relleno (incluso con 40 kHz de desvío) y el PER queda cerca de la teoría. Por eso los valores por defecto son **FLL 0,005, Costas 0,01 y sincronismo 0,01**. Se pueden cambiar durante la ejecución.

8.3 Indicadores del receptor

- **Eb/N0 estimado:** viene del bloque estándar `MPSK SNR Estimator`, con el método de 2.º y 4.º momento, aplicado a la salida de Costas. Se mide Es/N0 y se resta 3 dB, porque cada símbolo lleva 2 bits. Con el transmisor apagado muestra "sin señal".
- **PER teórico:** `PER = 1 - (1 - p)^(8·(N+18))`, con `p = Q(√(2·Eb/N0))` y tramas de N + 18 = 218 bytes. Usa `p` y no el BER (`2p(1–p)`) porque, con codificación diferencial, cada error del canal arruina 2 bits seguidos **de la misma trama**. Con el BER se contaría dos veces cada error y el PER teórico saldría casi el doble.
- **Desvío de frecuencia:** es la suma de lo que corrigen el FLL y Costas, promediada durante 0,1 s.
- **Tramas detectadas / con CRC correcto:** se cuentan con `Function Probe` sobre la cantidad de datos que sacó cada bloque (`nitems_written`).

9. Validación sin hardware

`tests/test_hackrf.py` usa los dos `.grc` reales y un canal de laboratorio simulado. Resultados:

Condición	Resultado
Desvío +15 kHz, relojes +20 ppm, Es/N0 = 22 dB, 2 pasadas	126 de 126 paquetes; archivo idéntico; desvío medido con error < 2 %
Desvío ±40 kHz (el peor caso entre dos HackRF), Es/N0 = 25 dB	0 paquetes perdidos; desvío medido 39 902 Hz y −40 054 Hz
Desvío ±40 kHz, Es/N0 = 12 dB (Eb/N0 = 9 dB)	PER 0,063 y 0,090 (teórico 0,058); con 3 o 4 pasadas el archivo llega idéntico
Es/N0 = 10 dB (Eb/N0 = 7 dB)	PER 0,80 – 0,85 (teórico 0,745): unos 0,3 dB peor que la teoría
Nombre de archivo largo con acentos; archivo inexistente	Se recorta el nombre; si no se encuentra el archivo, se transmite un aviso

Capturas con el HackRF **simulado** (desvío de 12 kHz, Es/N0 = 22 dB), para saber cómo se ve cuando todo funciona:

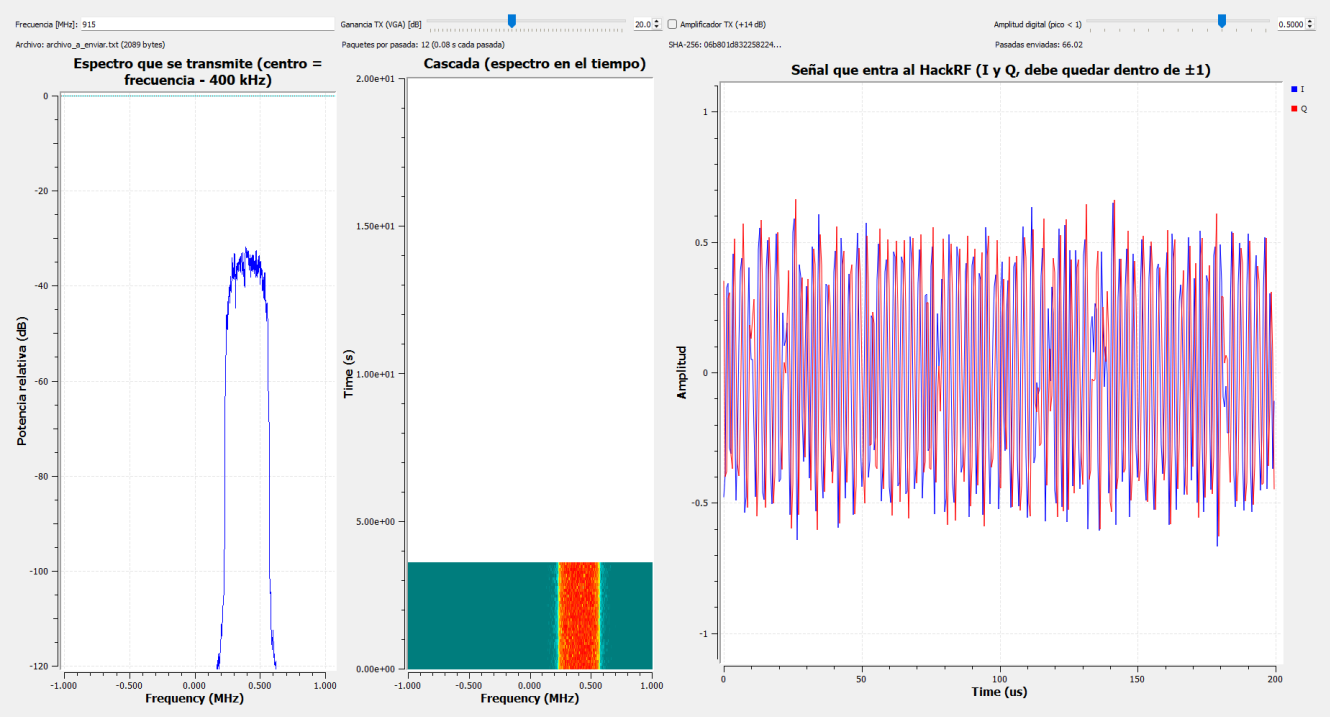


Figura 1 – Transmisor: espectro (la señal está en +400 kHz respecto de la frecuencia sintonizada), cascada y la señal que entra al HackRF, dentro de ±1.

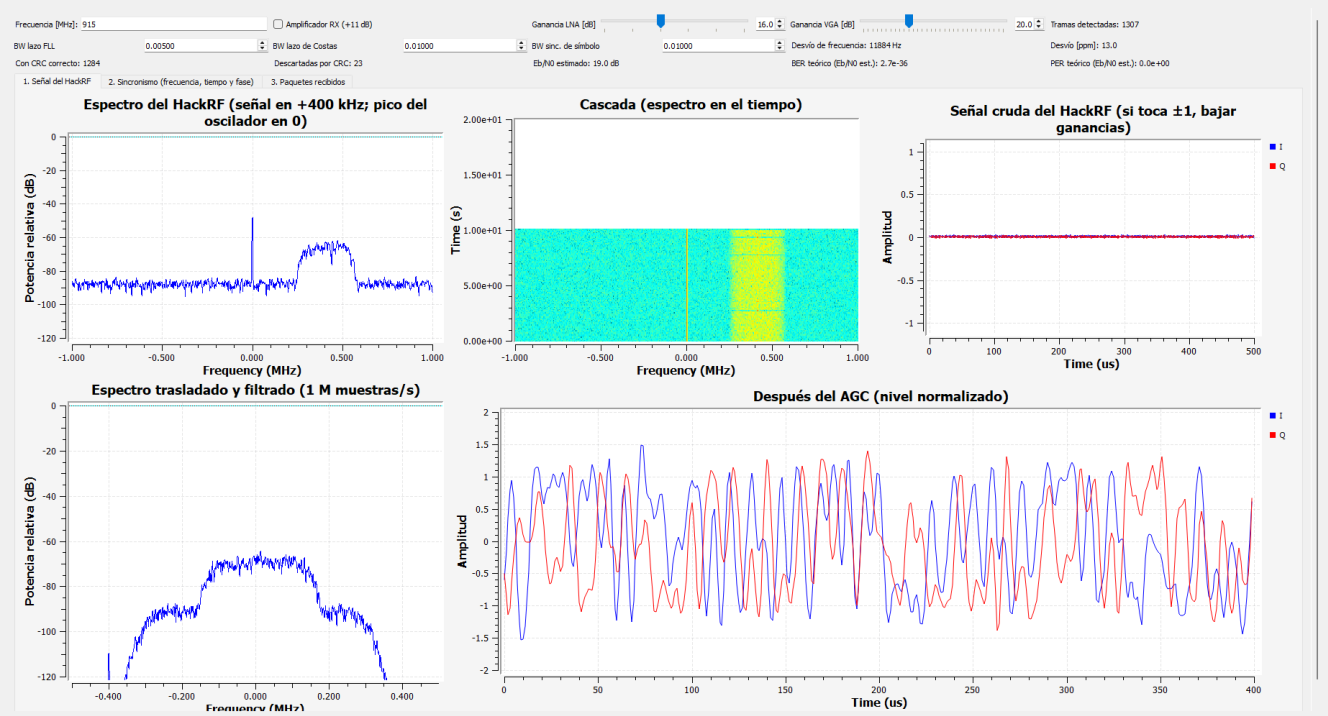


Figura 2 – Receptor, pestaña 1: espectro del HackRF (espiga de la portadora residual en 0 y señal en +400 kHz), cascada, señal cruda, espectro ya centrado y filtrado, y salida del AGC.

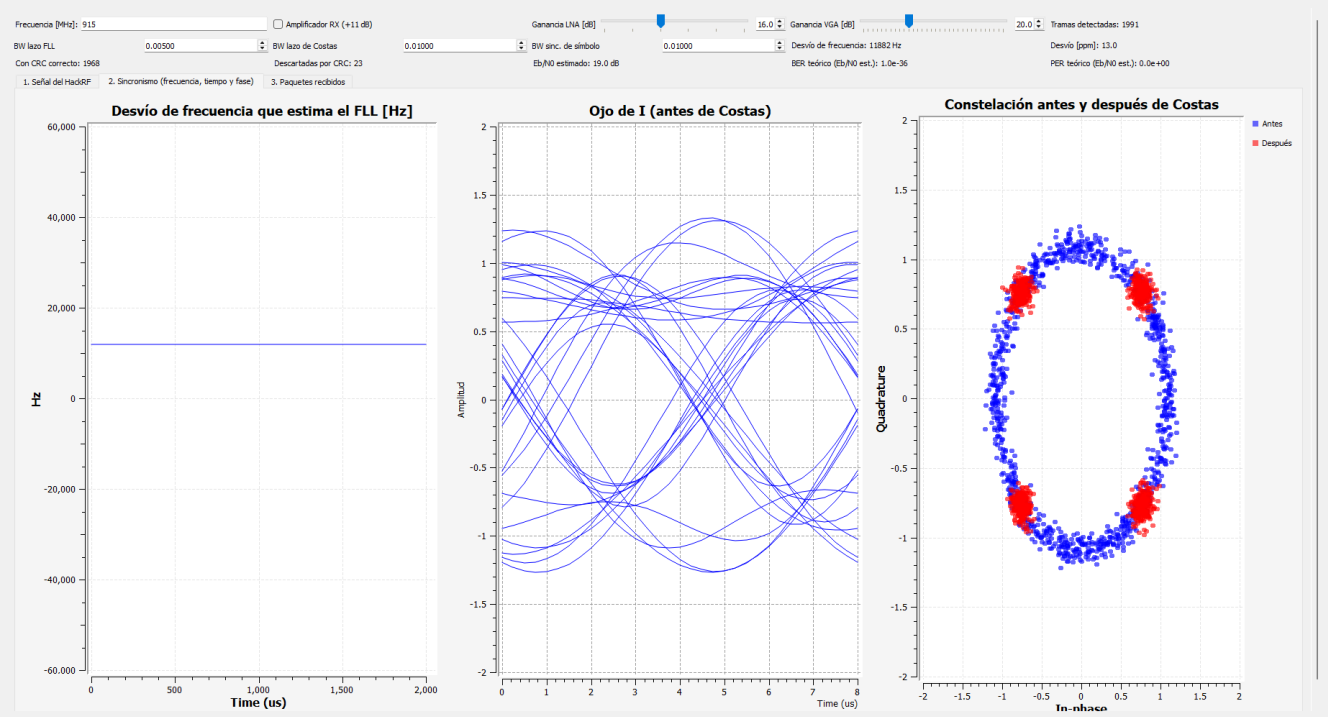


Figura 3 – Receptor, pestaña 2: desvío estimado por el FLL (unos 12 kHz), ojo antes de Costas y constelación antes (anillo: la fase todavía gira) y después de Costas (4 nubes). Las tramas descartadas por CRC de esta captura vienen del corte de la grabación simulada, que se repite en bucle.

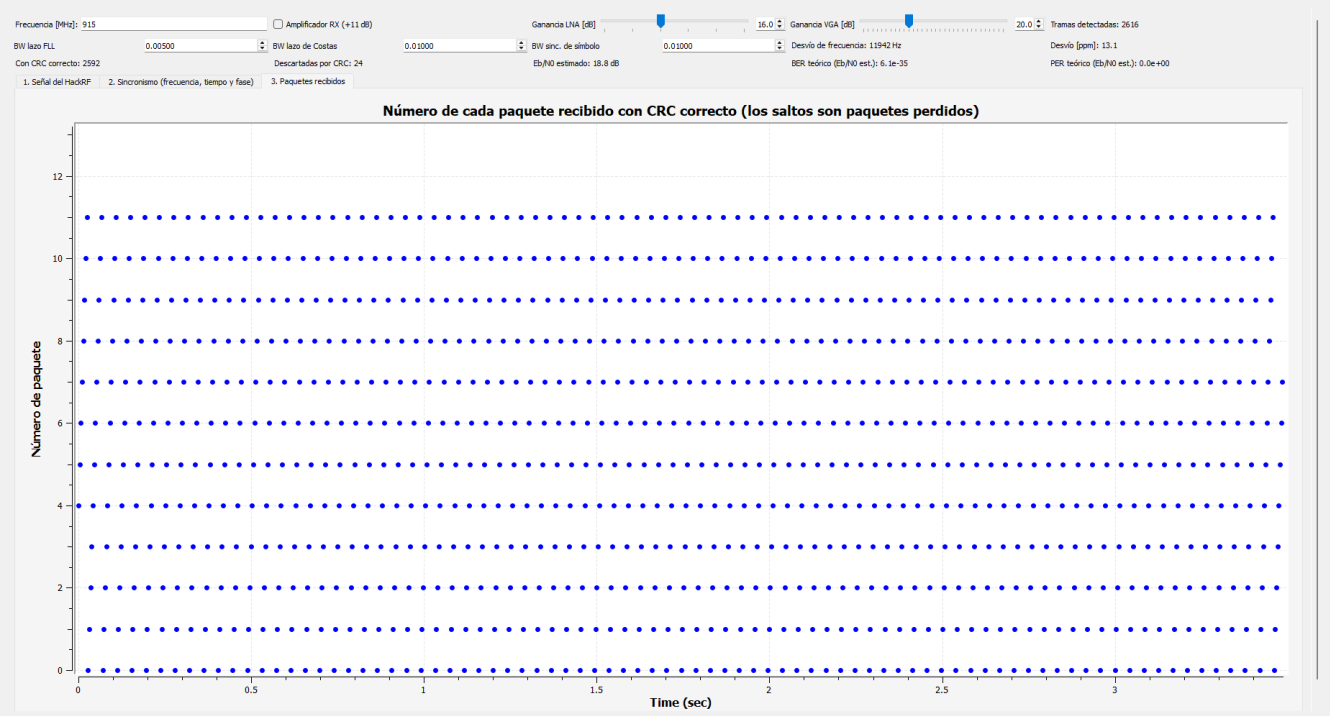


Figura 4 – Receptor, pestaña 3: número de cada paquete recibido (archivo de 12 paquetes). Están todas las filas completas, así que no se perdió ningún paquete.

10. Problemas comunes

Síntoma	Qué hacer
`hackrf_info` dice "No HackRF boards found"	Cable o puerto USB; driver WinUSB (Zadig) si Windows no lo instaló
Al ejecutar un `.grc`: `RuntimeError: SoapySDR::Device::make() no match`	GNU Radio no encontró la placa: revisar `hackrf_info`; que no la esté usando otro programa; con dos placas en la PC, que el `serial=` de `dispositivo` sea el correcto
Error al abrir el HackRF con dos placas en la misma PC	Poner el `serial=...` de cada una en la variable `dispositivo` (sección 5)
No se ve la señal en el espectro del RX	Misma frecuencia en TX y RX; antena en el conector *ANTENNA*; subir ganancias (6.3)
La señal cruda toca ±1 o la constelación es una mancha	Saturación: bajar la VGA o el LNA del RX, o la VGA del TX
Hay tramas detectadas pero muchas descartadas por CRC	Señal débil o saturada: revisar 6.3
El desvío medido pasa de unos 40 kHz y no se reciben tramas	El filtro de canal solo deja pasar ±40 kHz de desvío: corregir la frecuencia del RX en esa diferencia (por ejemplo, 915,050 MHz), o compartir el reloj entre las placas (prueba 4)
En la consola aparecen muchas `U` (TX) u `O` (RX)	La PC no da abasto: cerrar otros programas o usar una PC por HackRF
`reconstruir_archivo.py` dice que faltan paquetes	Transmitir más pasadas (`repeticiones`) o mejorar la señal
`reconstruir_archivo.py` pide `--n`	`N` (bytes por paquete) tiene que ser igual en los dos `.grc`

Archivos que genera el receptor (en la carpeta del `.grc`):

Archivo	Contenido
`paquetes_recibidos.bin`	Paquetes con CRC correcto, uno detrás de otro: [número (2 bytes) | 200 bytes]

Archivo	Contenido
`datos_recibidos.bin`	Solo los datos, en el orden en que llegaron. Con buena señal empieza con el archivo completo (después aparecen el paquete de descripción y las demás pasadas)
`recibido_<nombre>`	El archivo reconstruido por `reconstruir_archivo.py`
`grabacion_iq.cf32`	La señal cruda, solo si se habilitó `grabar_iq`